

Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning

Low-Level Design (LLD)

Title Page

Field	Value
Document Title	Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning - LLD
Document ID	HCL-PSAAI-LLD-001
Version	1.1
Date	2026-08-19
Author	Codex Documentation Review
Company	HCL
Source Document	Product_Support_Agentic_AI_Enterprise_Documentation.docx

Revision History

Version No	Date	Prepared by / Modified by	Significant Changes
1.0	2026-06-13	Senior Software Designer	Initial Software Low-Level Design specification
1.1	2026-08-19	Codex Documentation Review	Reviewed and aligned the LLD to the current repository classes, modules, and storage layout

Table of Contents

- 1. Overview
- 2. Project Structure
- 3. Core Models and Data Structures
- 4. Ingestion and Indexing Modules
- 5. Retrieval and RAG Modules
- 6. LLM Provider Layer
- 7. UI Layer
- 8. Algorithms and Processing Details
- 9. Error Handling and Logging
- 10. Configuration
- 11. Testing

- 12. SRS Traceability Notes
- 13. Documentation Consistency Notes
- 14. References

1. Overview

This LLD describes the implementation-level structure of the current codebase: modules, classes, responsibilities, key methods, data structures, and processing logic. It reflects the repository as of the reviewed state and avoids speculative future design.

2. Project Structure

```
app.py
src/product_support_agent/
|-- config.py
|-- core/
|   |-- bootstrap.py
|-- exceptions.py
|-- logger.py
|-- models/
|   |-- schemas.py
|-- services/
|   |-- chunking.py
|   |-- document_loader.py
|   |-- embedding_service.py
|   |-- gemini_service.py
|   |-- indexing_pipeline.py
|   |-- knowledge_base_service.py
|   |-- memory.py
|   |-- metadata_manager.py
|   |-- prompt_manager.py
|   |-- query_contextualizer.py
|   |-- rag_service.py
|   |-- retriever.py
|   |-- upload_manager.py
|   |-- vector_store.py
|   |-- llm/
|       |-- __init__.py
|       |-- base.py
|       |-- ollama_service.py
|-- loaders/
|   |-- base.py
|   |-- csv_loader.py
|   |-- pdf_loader.py
|   |-- txt_loader.py
`-- ui/
    |-- layout.py
    |-- presenter.py
    |-- sections.py
    |-- sidebar.py
    |-- theme.py
```

3. Core Models and Data Structures

Defined in [schemas.py](file:///D:/hcl/src/product_support_agent/models/schemas.py).

3.1 Key Models

Model	Purpose
UploadPayload	In-memory uploaded file bytes and filename
UploadResult	Result of save/skip behavior for one uploaded file

Model	Purpose
ExtractedDocument	Logical extracted unit from a file, page, or CSV row
ChunkMetadata	Metadata attached to each chunk
ChunkRecord	Chunk text plus chunk metadata
RetrievalResult	Ranked retrieval output with metadata and diagnostics
SourceReference	Deterministic citation model
DocumentEvidenceSummary	Per-document summary for multi-document reasoning
RAGResponse	Final response contract returned to the UI
ConversationMessage / ConversationState	Session-scoped memory state

3.2 Chunk Metadata Fields

The chunk metadata model includes:

- `file_name`
- `document_type`
- `chunk_number`
- `source_path`
- `timestamp`
- `page_number` when available
- `row_number` when available
- `section_title` when available
- `vector_id` after persistence

4. Ingestion and Indexing Modules

4.1 UploadManager

File: [upload_manager.py](file:///D:/hcl/src/product_support_agent/services/upload_manager.py)

Responsibilities:

- validate supported file extensions
- validate non-empty file content
- validate maximum upload size
- compute SHA-256 checksum directly from upload bytes
- skip checksum duplicates using the persistent manifest
- persist accepted files under `data/uploads`
- apply rename-on-collision only for same filename with different content

Important methods:

- `validate(payload)`
- `save_files(payloads)`
- `save_file(payload, checksum=None, manifest_lookup_required=True)`

4.2 DocumentLoader

File: [document_loader.py](file:///D:/hcl/src/product_support_agent/services/document_loader.py)

Responsibilities:

- choose the correct loader by extension
- aggregate extracted logical documents
- continue ingestion when some files fail extraction

Specialized loaders:

- `PDFDocumentLoader`
- `TXTDocumentLoader`
- `CSVDocumentLoader`

4.3 PDFDocumentLoader

File: [pdf_loader.py](file:///D:/hcl/src/product_support_agent/services/loaders/pdf_loader.py)

Implementation notes:

- uses `pypdf.PdfReader`
- extracts page by page
- tries layout extraction first
- normalizes whitespace
- reshapes table-like rows into pipe-delimited text when detected
- emits one `ExtractedDocument` per non-empty PDF page

4.4 TXTDocumentLoader

File: [txt_loader.py](file:///D:/hcl/src/product_support_agent/services/loaders/txt_loader.py)

Implementation notes:

- reads UTF-8 text with replacement for decode issues
- emits one `ExtractedDocument` per text file

4.5 CSVDocumentLoader

File: [csv_loader.py](file:///D:/hcl/src/product_support_agent/services/loaders/csv_loader.py)

Implementation notes:

- loads CSV through `pandas`
- converts each non-empty row into `column: value` text
- emits one `ExtractedDocument` per non-empty row

4.6 ChunkingService

File: [chunking.py](file:///D:/hcl/src/product_support_agent/services/chunking.py)

Responsibilities:

- generate metadata-rich chunks
- preserve section titles where possible
- keep table-like content coherent where possible
- apply configurable overlap

Key implementation details:

- uses `RecursiveCharacterTextSplitter`
- default separators: `["\n\n", "\n", ". ", " ", ""]`
- classifies lines as `table`, `bullet`, or `prose`
- detects uppercase headings and pipe-delimited chapter headings
- uses chunk ID format `file:page:row:chunk`

4.7 EmbeddingService

File: [embedding_service.py](file:///D:/hcl/src/product_support_agent/services/embedding_service.py)

Responsibilities:

- lazy-load Sentence Transformers model
- generate normalized numpy embeddings
- return `float32` vectors

Current model configuration is read from `config.py`; the local default is `all-MiniLM-L6-v2`.

4.8 VectorStoreService

File: [vector_store.py](file:///D:/hcl/src/product_support_agent/services/vector_store.py)

Responsibilities:

- load or create FAISS index
- validate embedding dimension consistency
- append embeddings
- persist index and vector metadata
- run similarity search

Implementation details:

- uses `faiss.IndexFlatIP`
- persists:
 - `data/faiss/index.faiss`
 - `data/faiss/index.pkl`

4.9 MetadataManager

File: [metadata_manager.py](file:///D:/hcl/src/product_support_agent/services/metadata_manager.py)

Responsibilities:

- maintain upload manifest
- maintain vector-state pickle metadata

- find known checksums
- attach `vector_id` values to persisted chunk metadata

Persistent manifest:

- `data/metadata/uploaded_files.json`

4.10 KnowledgeBaseIngestionPipeline

File: [indexing_pipeline.py](file:///D:/hcl/src/product_support_agent/services/indexing_pipeline.py)

Pipeline sequence:

1. ensure storage files
2. save uploads through `UploadManager`
3. record accepted uploads
4. extract logical documents
5. generate chunks
6. generate embeddings
7. persist vectors
8. return `IndexBuildResult`

4.11 KnowledgeBaseService

File: [knowledge_base_service.py]

(file:///D:/hcl/src/product_support_agent/services/knowledge_base_service.py)

Responsibility:

- clear local knowledge-base state by deleting contents of upload, FAISS, and metadata directories
- recreate required directories and reset the manifest

5. Retrieval and RAG Modules

5.1 RetrieverService

File: [retriever.py](file:///D:/hcl/src/product_support_agent/services/retriever.py)

Responsibilities:

- validate query text and top-k
- generate local query embeddings
- search FAISS
- build retrieval result objects
- rerank and filter weak structural matches
- suppress duplicate retrieval results
- expand adjacent chunks to improve context completeness

Rerank inputs include:

- semantic score
- term overlap

- section-title overlap
- exact phrase overlap
- structural penalties

5.2 ConversationMemoryService

File: [memory.py](file:///D:/hcl/src/product_support_agent/services/memory.py)

Responsibilities:

- store session-scoped `ConversationState`
- add user and assistant turns
- return complete history
- return filtered recent history used for contextualization
- clear history

Important behavior:

- only grounded assistant responses are included in `get_recent_history()`
- recent history is capped by `memory_max_turns`

5.3 QueryContextualizer

File: [query_contextualizer.py](file:///D:/hcl/src/product_support_agent/services/query_contextualizer.py)

Responsibilities:

- normalize the current query
- load the contextualization prompt
- format conversation history
- call the configured LLM provider for rewriting
- fall back to the original query if rewriting fails or returns empty text

5.4 PromptManager

File: [prompt_manager.py](file:///D:/hcl/src/product_support_agent/services/prompt_manager.py)

Responsibilities:

- ensure prompt asset files exist
- load prompt assets from `prompts/`

Managed prompt files:

- `system_prompt.txt`
- `query_contextualization_prompt.txt`
- `document_summary_prompt.txt`
- `multi_document_synthesis_prompt.txt`

5.5 RAGService

File: [rag_service.py](file:///D:/hcl/src/product_support_agent/services/rag_service.py)

Responsibilities:

- public entry point for question answering
- build and execute the LangGraph workflow
- apply relevance filtering
- choose reasoning strategy
- coordinate provider calls
- build deterministic citations
- update conversation memory

Implemented LangGraph nodes:

1. `load_memory`
2. `contextualize_query`
3. `retrieve_chunks`
4. `no_results_fallback`
5. `filter_relevance`
6. `low_relevance_fallback`
7. `determine_reasoning_strategy`
8. `simple_grounded_answer`
9. `summarize_document_evidence`
10. `synthesize_multi_document_answer`
11. `format_citations`
12. `update_memory`

6. LLM Provider Layer

6.1 `BaseLLMService`

File: [base.py](file:///D:/hcl/src/product_support_agent/services/llm/base.py)

Provider contract methods:

- `contextualize_query()`
- `generate_grounded_answer()`
- `summarize_document()`
- `synthesize_documents()`

6.2 `OllamaService`

File: [ollama_service.py](file:///D:/hcl/src/product_support_agent/services/llm/ollama_service.py)

Responsibilities:

- call the official Ollama Python client
- issue chat requests to the configured local model
- treat non-success completion states as failures
- map provider/network failures into project exceptions

6.3 `GeminiService` and `GeminiLLMAdapter`

Files:

- [gemini_service.py](file:///D:/hcl/src/product_support_agent/services/gemini_service.py)
- [init.py](file:///D:/hcl/src/product_support_agent/services/llm/init.py)

Responsibilities:

- keep Gemini support available in the repository
- enforce finish-reason validation
- expose Gemini through the same `BaseLLMService` contract via adapter

6.4 Provider Resolution

`build_llm_service(settings)` resolves the active provider from environment settings. The current local environment is configured for `ollama` with model `qwen3:8b`.

7. UI Layer

7.1 `layout.py`

Sets page configuration, applies enterprise theme, renders sidebar, header metrics, and primary segmented navigation.

7.2 `sections.py`

Contains the main user workflows:

- knowledge-base upload/build/clear
- Ask AI form submission
- conversation rendering
- response handling

7.3 `sidebar.py`

Renders:

- workspace summary
- AI status
- conversation controls
- developer tools for latest answer inspection

7.4 `presenter.py`

Provides:

- sidebar snapshot derivation
- provider connectivity check
- session transcript helpers
- response-time formatting

7.5 `theme.py`

Contains:

- custom Streamlit CSS theme
- shell header renderer
- message-bubble renderers
- reusable card renderers

8. Algorithms and Processing Details

8.1 Chunking

Chunking uses:

- semantic block construction
- recursive splitting for oversized blocks
- overlap block carry-forward for prose content
- stable chunk numbering

8.2 Retrieval

Retrieval uses:

- normalized local embeddings
- FAISS candidate search
- reranking by lexical and structural signals
- result deduplication
- adjacent chunk expansion
- relevance-threshold filtering in `RAGService`

8.3 Fallback Rules

The core insufficient-evidence fallback string is:

`I could not find enough information in the available knowledge base to answer this question.`

If generation is unavailable, `RAGService` records:

- `grounded=False`
- `message="Grounded answer generation is currently unavailable."`

9. Error Handling and Logging

9.1 Exception Families

Defined in [exceptions.py](file:///D:/hcl/src/product_support_agent/exceptions.py), including:

- `ValidationError`
- `ExtractionError`
- `EmbeddingError`
- `ConfigurationError`

- `GenerationError`
- `ContextualizationError`
- `IndexPersistenceError`
- `MetadataError`
- `IngestionPipelineError`

9.2 Logging

Configured in [logger.py](file:///D:/hcl/src/product_support_agent/logger.py):

- log file: `logs/product_support_agent.log`
- rotating file handler
- structured format with timestamp, level, logger name, and message

10. Configuration

All runtime configuration is centralized in [config.py](file:///D:/hcl/src/product_support_agent/config.py).

Important configuration groups:

- paths
- logging
- chunking
- retrieval
- memory
- embedding model
- Gemini settings
- Ollama settings
- provider selection

The current implementation uses `.env` plus code defaults. YAML configuration is not implemented in the runtime code.

11. Testing

The repository includes automated tests for:

- upload manager
- indexing pipeline
- vector store persistence
- chunking behavior
- retriever behavior
- RAG service behavior
- memory service behavior
- knowledge-base clearing

Test design is implementation-aligned rather than documentation-only.

12. SRS Traceability Notes

LLD Area	Related SRS Focus
upload validation and checksum dedupe	FR-001 to FR-004
extraction and metadata	FR-005, FR-006
chunking, embeddings, persistence	FR-007 to FR-009
retrieval, reranking, expansion	FR-010, FR-011
query rewriting and memory	FR-012, FR-013
grounded RAG and citations	FR-014 to FR-016
multi-document reasoning	FR-017
knowledge-base and session clearing	FR-018
diagnostics and logging	FR-019, FR-020

13. Documentation Consistency Notes

- This reviewed LLD removes the older YAML-config and Gemini-only assumptions from the earlier draft.
- The current repository uses a provider abstraction and supports both `ollama` and `gemini`.
- The active local environment selects `ollama` with `qwen3:8b`, but the code default remains `gemini` if no provider is set.
- Browser-only UI claims, GPU routing guarantees, and hard benchmark numbers are not documented here as verified implementation facts unless directly supported by the repository.

14. References

1. [app.py](file:///D:/hcl/app.py)
2. [config.py](file:///D:/hcl/src/product_support_agent/config.py)
3. [schemas.py](file:///D:/hcl/src/product_support_agent/models/schemas.py)
4. [indexing_pipeline.py](file:///D:/hcl/src/product_support_agent/services/indexing_pipeline.py)
5. [retriever.py](file:///D:/hcl/src/product_support_agent/services/retriever.py)
6. [rag_service.py](file:///D:/hcl/src/product_support_agent/services/rag_service.py)
7. [sections.py](file:///D:/hcl/src/product_support_agent/ui/sections.py)
8. [README.md](file:///D:/hcl/README.md)